ChatGPT

Analytical Review of the Coinbase RTB Bot and Alignment With the Research Standards

Executive summary

The Coinbase [1] RTB bot repository is a deliberately **single-process, async, SQLite-backed** trading system with a strong emphasis on **paper-mode orchestration**, deterministic fill simulation, and restart-safe persistence. Its core runtime (`main.py`) wires together: a Coinbase Advanced Trade WebSocket market-data feed, a trade-to-bar builder, a rolling bar aggregator, a breakout–retest state machine that emits signals to SQLite, an execution/reconciliation loop that creates orders and positions, safeguard checks, and structured event logging. [2] [3]

Against the two user-provided documents, the biggest mismatch is *directional*: the `research_plan.md` concludes the breakout–retest–continuation concept is **archived** after producing too few trades, while the repo's `bot/state_machine.py` still hard-codes that concept and its parameters. [4] filecite turn0file1

The second mismatch is *capability*: the research standards demand a **credible backtest + walk-forward + parameter sensitivity + multiple-testing defense** pipeline. The repo, as it stands, provides a solid **paper/runtime harness**, not a full research/backtest platform (the roadmap mentions replay/walk-forward, but no replay/backtest engine exists in the code tree). [5] filecite turn0file1

This report therefore treats the repository as a **usable execution/paper scaffold** that needs (a) **strategy modularization** and (b) a **research-grade evaluation subsystem** to satisfy the pasted standards, especially if you take the "pick a lane" directive seriously (Lane A/B/C decision and feasibility matrix before further strategy coding). filecite turn0file0

Summary of the two user-provided documents

Research plan and backtesting standards

The `research_plan.md` is a governance document defining what counts as "validated" in a crypto strategy pipeline. It sets hard **gates** for backtest → paper → live, with explicit "do not proceed" thresholds based on trade count, expectancy, walk-forward robustness, and parameter sensitivity. filecite turn0file1

Its key requirements (interpreting all undefined external artifacts as **unspecified**) include:

- **Minimum sample sizes**: a "floor" of ~30 trades/year for even minimal evaluation; stronger targets at ~100 trades; and a "research-grade" standard referencing ~300 trades. (The plan states the current breakout strategy produces only ~1 trade in 365 days and fails these gates.) filecite turn0file1
- **Execution realism**: backtests must include transaction-cost modeling, slippage assumptions, and no lookahead bias; 4h bars should be derived from 1h bars (not fetched separately). filecite turn0file1

- **Validation methodology**: walk-forward analysis, parameter sensitivity sweeps (±20% on each parameter), and multiple-testing / overfitting defenses are required before trust. filecite turn0file1
- **Operational gates**: paper trading for weeks with quantitative divergence checks before live capital is allowed, plus kill-switch / shutdown criteria. filecite turn0file1
- **Conclusion**: the breakout–retest–continuation concept is explicitly archived, and the plan points toward alternative strategy families after repeated failures. filecite turn0file1

Assumptions and constraints embedded in the plan include: BTC-USD as a reference dataset; hourly bars as the main decision interval; and a strong assumption that statistical significance requires high trade counts that the current concept cannot generate (hence it is "archived"). filecite turn0file1

Ambiguities that matter for implementation include: - Several referenced files are not present here (`signal_funnel_audit_results.md`, VPMR diagnostics, "Research Pack" contents), so any requirement that depends on them is **unspecified** beyond the summarized thresholds. filecite turn0file1
- Fee/slippage numbers are presented as "current model" but not tied to a specific exchange fee tier or liquidity regime; treating them as stable constants is likely brittle without a data-backed calibration loop. (This is a methodological ambiguity rather than a code one.) filecite turn0file1

Lane selection directive and feasibility-matrix requirement

The `Pasted text.txt` is a strategic correction memo. Its core claim is that "the next strategy" should not be the unit of progress; instead, progress is a **capability-to-edge match**. filecite turn0file0

Its key requirements: - Treat the prior "Research Pack" as **validation standards**, not as an idea generator. filecite turn0file0
- Choose among lanes before building more strategy code: - **Lane A**: OHLCV-only, spot execution, multi-asset rule library, longer horizons (daily/4h) with multiple-testing control, walk-forward evaluation, and portfolios/ensembles. - **Lane B**: derivatives carry/basis/funding strategies—mechanism-backed but requires derivative access and introduces margin/liquidation/exchange risks. - **Lane C**: order-book / order-flow / microstructure signals—arguably most "real," but heavy engineering and data requirements. - **Lane D**: low priority: yet another BTC-only indicator strategy (only as a cheap pretest). filecite turn0file0
- Add a missing step: build a **feasibility matrix** mapping strategy families to required data, instruments, latency, operational complexity, and expected trade count. filecite turn0file0

Ambiguities: - The memo references "the paper showing selected portfolios of simple rules can work out of sample" and another paper about hourly vs daily technical analysis strength, but does not name them. This matters because the repo's next technical direction depends on what those papers actually show. filecite turn0file0

In the web literature, there is support for the general idea that **large libraries of technical rules** plus **multiple-testing defenses** and sometimes **portfolio construction** can matter, while single-rule/single-asset conclusions can be fragile. For example, Hudson & Urquhart examine a very large rule set and explicitly use multiple-hypothesis procedures; they report mixed out-of-sample results for Bitcoin depending on period/specification. [6] The more recent Frömmel & Deprez study explicitly constructs portfolios of selected rules across daily and intraday sets and reports out-of-sample outperformance in their design. [7] These papers do not prove "Lane A will work," but they do strengthen the memo's claim

that if technical rules work, it may be via **breadth + selection discipline**, not a single handcrafted BTC-only chain. [8]

Repository analysis

Repository structure and intent

The repository's own documentation positions CB-RTB v0 as an explicitly monolithic, single-process async system, initially designed for BTC-USD, with documentation for strategy rules, risk management, and execution constraints. [9]

The effective "production" stack is the `bot/` package plus the root `main.py`, while the `src/` directory appears to be a legacy/abandoned scaffold ("src/ is not imported" is stated directly in runtime entrypoint docs). [2] [10]

Key top-level artifacts: - `main.py`: only supported runtime entrypoint for paper mode; wires the bot end-to-end and runs three async tasks. [2]
- `config.yaml`: runtime + strategy + risk + execution settings; however, several modules do not yet consume these settings (notably `bot/state_machine.py` and `bot/risk.py`). [11] [4] [12]
- `requirements.txt`: runtime deps include `websockets`, `aiohttp`, `pyyaml`, `coinbase-advanced-py`; test tooling (e.g., pytest) is not listed here. [13] [14]

The included docs show that paper mode integration was intentionally scoped to orchestration/validation, not profitability validation, with deterministic synthetic fills and operator safeguards. [15] [16]

Runtime architecture, modules, and data flow

The runtime is a single asyncio event loop with three tasks: - Market data loop (`MarketDataProcessor.run`) reading from a WebSocket-backed queue - Signal consumer loop that (a) processes NEW signals into orders (if safeguards allow) and (b) runs reconciliation every interval - Safeguard monitor loop that periodically evaluates guards and logs disable events [2]

The core internal flow is:

```
flowchart TD
  WS[Coinbase WS: market_trades + heartbeats] --> CA[CoinbaseAdapter.ws_loop]
  CA --> MQ[market_queue asyncio.Queue]
  MQ --> MDP[MarketDataProcessor.run]
  MDP --> BB[BarBuilder.process_trade]
  BB -->|closed bars| OBC[on_bar_close callback]
  OBC --> J1[Journal.upsert_bar]
  OBC --> AGG[BarAggregator.add]
  AGG --> SM[StateMachine.process_bars]
  SM -->|INSERT| SIG[(SQLite: signals)]
  SC[signal_consumer_task] -->|SELECT status=NEW| SIG
  SC --> EXE[ExecutionService.process_signal]
```

```
EXE --> ORD[(SQLite: orders)]
SC -->|reconcile| REC[ExecutionService.reconcile_pending_orders]
REC --> AD[PaperAdapter or future live adapter]
AD -->|fills| HF[ExecutionService.handle_fill]
HF --> EXC[(SQLite: executions)]
HF --> POS[(SQLite: positions)]
SG[Safeguards] --> RS[(SQLite: runtime_state)]
EV[log_event] --> EL[(SQLite: event_log)]
```

This is implemented concretely in `main.py` plus the `bot/` modules: - Market data processing and bar building: `bot/market_data.py`, `bot/bars.py` [17] [18]
- Aggregation bridge and warm-start from DB: `bot/aggregator.py` [19]
- Signal state machine and rule logic: `bot/state_machine.py`, `bot/strategy.py` [4] [20]
- Persistence and journaling: `bot/db.py`, `bot/journal.py` [21] [22]
- Execution/reconciliation + risk sizing: `bot/execution.py`, `bot/risk.py` [23] [12]
- Paper-mode adapter and deterministic fill behavior: `bot/adapters.py` [24]
- Safety + event logs: `bot/safeguards.py`, `bot/events.py` [25] [26]

External APIs, dependencies, and exchange semantics

The bot uses the "Advanced Trade" WebSocket endpoint (`wss://advanced-trade-ws.coinbase.com`) and subscribes to `market_trades` and `heartbeats`, plus `user` when authenticated. [27] The official Coinbase [1] developer docs enumerate these channels (including `level2`, `candles`, etc.) and explicitly recommend heartbeats because "most channels close within 60–90 seconds if no updates are sent." [28] This aligns with the bot subscribing to heartbeats in `CoinbaseAdapter.ws_loop`. [27]

The docs also state that WebSocket JWTs expire after **2 minutes**. [29] The adapter's reconnection logic (refreshing after ~115 seconds) is consistent with that constraint. [27]

Rate limits for WebSocket connections/messages are stated as **8 per second per IP address**. [30] This is operationally relevant if you implement multi-symbol or multiple channels (Lane C), because the bot may need to consolidate subscriptions into a smaller number of sockets and manage message throughput.

For historical data and Lane A style work, the Advanced Trade REST API supports **product candles** with granularities up to `ONE_DAY`, including `ONE_HOUR` and `FOUR_HOUR`, though with server-side limits (e.g., "max 350" candles per request). [31] This is directly relevant to building a backtest harness and walk-forward engine without relying on trade-level replay.

Persistence model and restart safety

SQLite tables include: - `bars` keyed by `(symbol, timeframe, ts_open)` with upsert semantics - `signals`, `orders`, `executions`, `positions` - `event_log` for structured lifecycle logging - `runtime_state` for persisted state (state machine + safeguards) [21]

A key architectural strength is that both the strategy process and safety state persist across restarts: - `StateMachine` loads an `algo_state` JSON blob from `runtime_state` and rehydrates internal fields.

[4]
- `Safeguards` persists `trading_enabled` and a `_tripped` set, so safety disables are sticky. [25]

Tests and current deployment posture

The test suite is non-trivial and covers: - DB upserts and state survival (`test_db.py`) - Config validation, including "live mode exits" (`test_config.py`) - State machine lifecycle and restart persistence (`test_state_machine.py`) - Execution idempotency, partial fills handling, and overfill prevention (`test_execution.py`) - Reconciliation paths: timeout, failed exchange, adapter-driven fills, duplicate fill idempotency (`test_reconcile.py`) - Paper-mode E2E lifecycle and deterministic fill behavior (`test_paper_mode.py`) - Safeguards behavior and persistence (`test_safeguards.py`) - Main loop event emission helpers (`test_main_events.py`) [32] [33] [34] [35] [36] [37] [38] [39]

There is no evident CI workflow in-repo (no `.github/workflows` found via connector search), so running tests is currently a manual discipline. (This is an inferred gap; absence of evidence in visible tree.)

Requirements-to-code mapping and gap analysis

Mapping table

The table below maps the *user-provided* requirements to locations in the repo where they are implemented, partially implemented, or absent.

Requirement source	Requirement (condensed)	Code / doc mapping	Status
`research_plan.md` filecite turn0file1	Walk-forward validation required before trust	Mentioned as roadmap (`bot/replay.py` planned) but no replay/backtest module present	Missing
`research_plan.md` filecite turn0file1	Parameter sensitivity sweeps (±20%) and logging	No parameter-sweep framework in repo	Missing
`research_plan.md` filecite turn0file1	Execution realism: fees + slippage, and realistic fills	Paper mode intentionally uses deterministic fills with `commission=0.0` (`PaperAdapter`); risk slippage is hard-coded 0.5%	Misaligned / partial
`research_plan.md` filecite turn0file1	Paper trading gates and quantitative go/no-go	Paper mode runtime exists; gates/reporting logic not implemented beyond event counts	Partial
`research_plan.md` filecite turn0file1	Kill-switch / daily loss limit	Safeguards supports disable + persistence; daily-loss guard is stubbed to False	Partial

Requirement source	Requirement (condensed)	Code / doc mapping	Status
`research_plan.md` filecite turn0file1	Strategy archived; do not proceed with breakout chain	Strategy is still the core `StateMachine` implementation	Misaligned
`Pasted text.txt` filecite turn0file0	Build feasibility matrix before more strategy code	No feasibility-matrix artifacts exist in repo	Missing
`Pasted text.txt` filecite turn0file0	Lane A: multi-asset OHLCV rule library + portfolios, daily/4h focus	Repo is single-symbol, single-strategy, with live WebSocket trade ingestion; can be extended using REST candles	Missing (needs refactor)
`Pasted text.txt` filecite turn0file0	Lane B: derivatives carry/basis/ funding	Repo targets spot Advanced Trade; no derivatives functionality, live mode disabled	Missing / out of scope
`Pasted text.txt` filecite turn0file0	Lane C: order-flow / L2 microstructure	WebSocket supports `level2` channel (external); repo currently subscribes only to `market_trades` / `heartbeats`	Missing (non-trivial)
Repo docs [15]	Paper mode must be deterministic and lifecycle-correct	`PaperAdapter`, `ExecutionService.reconcile_pending_orders`, tests in `test_paper_mode.py`	Implemented
Repo docs [15]	Single entrypoint and no `src/` imports	Root `main.py` uses `bot.*` only; `src/` remains unused	Implemented (with legacy debt)

Key code references supporting the table: the paper-mode integration spec/build report (design intent), the runtime entrypoint, the strategy state machine, the risk manager, and safeguards. [15] [16] [2] [4] [12] [25]

Gap analysis

The most important gaps, ordered by how directly they block the pasted research standards:

The repo has no research-grade evaluation loop. The pasted standards require walk-forward and sensitivity sweeps, plus multiple-testing defenses. The repo has neither a `replay.py` nor a backtest runner. Even the repo's own roadmap explicitly calls out a replay/walk-forward step but does not implement it. [5] 【filecite】turn0file1【】

The strategy layer is not modularized and remains locked to the archived breakout concept. The `StateMachine` encodes the breakout-retest logic and hard-coded thresholds, which is exactly the strategy family the pasted research plan says should be archived. [4] 【filecite】turn0file1【】

Risk and execution realism are inconsistent across config, docs, and code. `config.yaml` carries risk and slippage parameters, but `RiskManager` uses hard-coded `MAX_RISK_PERCENT=0.002` and `MAX_SLIPPAGE=0.005`, and paper fills use zero fees and fill-at-limit. This is consistent with the repo's paper-mode spec ("orchestration validation"), but inconsistent with the research-plan requirement that backtests model costs realistically and that paper results correlate with backtest projections. [11] [12] [24] 【filecite】turn0file1【】

The lane-selection directive is not represented in artifacts. There is no feasibility matrix, and the runtime architecture is optimized around a single symbol and a single state machine. If you accept the "capability-to-edge match" framing, this is a structural defect: you need a representation of capabilities (data, instruments, latency, etc.) and how they unlock strategy families. 【filecite】turn0file0【】

Prioritized recommendations and concrete implementation strategy

Prioritized task list with effort and risk

Priority	Recommendation	Why it matters vs. pasted requirements	Effort	Risk
Short-term	Treat `bot/state_machine.py` as an *optional plugin* and add a Strategy interface	Enables lane choice without rewriting the runtime every time; aligns with "capability-to-edge match"	Medium	Medium
Short-term	Make `RiskManager` and `StateMachine` parameters config-driven (no hard-coded constants)	Required for parameter sensitivity sweeps; reduces "hidden parameters"	Medium	Medium
Short-term	Add CI (pytest + minimal lint/type checks)	Reduces regression risk while refactoring for lane A/backtests	Low–Medium	Low
Short-term	Normalize data access patterns (DB warms should be `ORDER BY ts_open DESC LIMIT N`)	Prevents performance collapse as DB grows in long paper runs	Low	Low

Priority	Recommendation	Why it matters vs. pasted requirements	Effort	Risk
Medium-term	Build `replay/backtest` runner using REST “Get Product Candles” with 1h/4h/1d granularities	Directly satisfies walk-forward and trade-count gates; avoids needing trade-level replay	High	Medium–High
Medium-term	Implement a walk-forward harness + parameter sweep framework + results persistence	Direct implementation of the pasted validation gates	High	High
Medium-term	Implement realistic cost model module (fees, spread, slippage) and unify with config	Align paper/backtest realism requirements; supports “cost sensitivity” in feasibility matrix	Medium–High	Medium
Long-term	Multi-asset support + portfolio/ensemble evaluation (Lane A)	Matches the highest-ranked feasible lane if you remain OHLCV/spot-only	Very High	High
Long-term	Level2/order-flow ingestion (Lane C)	Enables microstructure research; requires very different data models and performance work	Very High	Very High
Long-term	Derivatives carry/basis (Lane B)	Requires derivative market access, risk controls, and likely a different adapter surface	High	High

Proposed implementation sequence and interfaces

This strategy assumes you accept the pasted lane memo’s core constraint: do not build “the next BTC-only strategy” until you have the feasibility matrix and the evaluation harness. filecite turn0file0

Implement feasibility matrix as a first-class artifact

Create a repository artifact (e.g., `docs/research/feasibility_matrix.yaml` + `docs/research/feasibility_matrix.md`) that encodes, for each lane: - required data (OHLCV vs candles vs L2) - instruments (spot vs perps/futures) - latency expectations - operational risks - expected trade frequency range - backtestability in this repo’s stack (Yes/No/Partial)

You can ground parts of this from published sources: - Lane A: evidence that very large rule libraries with multiple-testing defenses can show predictability, but Bitcoin results may be weaker than other coins and are period-dependent. [6]
- Lane B: “crypto carry” and basis are studied as structural features of spot–futures markets, with drivers connected to boom–bust dynamics; this supports the “mechanism-backed” claim. [40]
- Lane C: microstructure research on bitcoin/coinbase emphasizes the role of limit orders and book state in

price discovery; this supports the claim that L2/order-flow signals are information-rich, albeit engineering-heavy. [41]

Add a strategy plugin interface

Add `bot/strategy_base.py`:

```
from dataclasses import dataclass
from typing import Protocol, Sequence, Optional
from .models import Bar, Signal

@dataclass(frozen=True)
class StrategyContext:
    symbol: str
    portfolio_value: float

class Strategy(Protocol):
    def on_bar_close(
        self,
        bars_1h: Sequence[Bar],
        bars_4h: Sequence[Bar],
        ctx: StrategyContext,
    ) -> Optional[Signal]:
        ...
```

Then implement: - `BreakoutRetestStrategy` wrapping the current `StateMachine` logic (so it can be archived without deleting code). - Future Lane-A strategies as other implementations.

This isolates the runtime (queues, DB, reconcile loop) from "what generates signals."

Build a backtest/replay subsystem using REST candles

Use the Advanced Trade REST "Get Product Candles" endpoint, which supports `ONE_HOUR`, `FOUR_HOUR`, and `ONE_DAY` granularities. [31] This reduces dependence on trade-level replay and aligns better with Lane A's daily/4h emphasis. filecite turn0file0

A recommended architecture:

- `research/data/coinbase_candles.py`: downloader with pagination (since limit is capped; you must loop). [31]
- `research/backtest/engine.py`: event-driven simulator that iterates bars and calls `Strategy.on_bar_close`, then runs a simulated execution model (not paper adapter) that applies costs.
- `research/validation/walk_forward.py`: creates rolling windows and logs in-sample vs OOS deltas.

- `research/validation/multiple_testing.py`: implements at least one defense appropriate for rule libraries.

On multiple testing: White's Reality Check is specifically designed to address data snooping by testing whether the best model has superiority over a benchmark after a specification search. [42] Bailey et al.'s "Probability of Backtest Overfitting (PBO)" provides an explicit framework (CSCV) to estimate the probability a backtest is overfit. [43] Either (or both) can be used to operationalize the research-plan emphasis on guarding against curve-fitting.

Unify config consumption and parameterization

Refactor: - `bot/risk.py` to pull `risk_per_trade` and slippage constraints from `config.yaml` (via `bot/config.py`) instead of hard-coded constants. [12] [44]
- `bot/state_machine.py` to pull breakout thresholds from config instead of embedding them. [4] [11]

This change is prerequisite for a meaningful parameter sweep framework, because otherwise the sweep cannot "reach" the true parameters.

Expand observability into metrics usable for gates

Today's observability is the SQLite `event_log` + the printed session summary. [2] [26]

To support the research-plan gates, add: - a "run manifest" table (run_id, git_sha, config hash, start/end timestamps) - computed metrics tables (trade_count, PF, expectancy, max DD) for backtests and paper runs - divergence checks (paper vs backtest) as first-class outputs (rather than external spreadsheets)

Timeline and milestones

A concrete timeline (assuming one developer; adjust if team size differs):

```
gantt
  title CB-RTB alignment timeline
  dateFormat  YYYY-MM-DD
  axisFormat  %b %d

  section Foundation
  Add CI (pytest, lint, type checks)                :a1, 2026-04-06, 3d
  Refactor config consumption (risk + strategy params):a2, 2026-04-08, 5d

  section Lane decision artifacts
  Feasibility matrix (YAML+MD) + decision record     :b1, 2026-04-10, 4d

  section Research harness
  Candle downloader + local dataset cache            :c1, 2026-04-14, 5d
  Backtest engine (event-driven, cost model)         :c2, 2026-04-18, 10d
  Walk-forward runner + parameter sweep framework    :c3, 2026-04-28, 10d
```

```
  section Strategy modularization
  Strategy interface + breakout strategy wrapper     :d1, 2026-04-14, 6d
  Lane A prototype rule-library scaffold             :d2, 2026-04-22, 12d

  section Observability gates
  Metrics persistence + report generator             :e1, 2026-05-06, 7d
  Paper-vs-backtest divergence checks                :e2, 2026-05-10, 7d
```

Milestones: - “Repo-safe refactor” milestone: CI green + config-driven parameters. - “Lane commitment” milestone: feasibility matrix + explicit lane choice recorded. - “Validation-capable” milestone: walk-forward + sweep + multiple-testing defense producing machine-readable reports. - “Lane A usable” milestone: multi-asset dataset + rule evaluation + portfolio construction experiments.

Sample code/pseudocode for critical changes

Config-driven risk manager

```python
# bot/risk.py (sketch)
from . import config

class RiskManager:
    @staticmethod
    def risk_fraction() -> float:
        return float(config._raw.get("risk", {}).get("risk_per_trade", 0.002))

    @staticmethod
    def max_slippage_bps() -> float:
        return float(config._raw.get("execution",
{}).get("max_entry_slippage_bps", 10))

    @staticmethod
    def get_ioc_limit(signal_price: float) -> float:
        bps = RiskManager.max_slippage_bps()
        return round(signal_price * (1 + bps / 10_000.0), 2)
```

This directly aligns the implementation with the config surface already declared (`max_entry_slippage_bps`, etc.). [11]

Walk-forward runner skeleton

```python
# research/validation/walk_forward.py (sketch)
from dataclasses import dataclass
from typing import Iterable

@dataclass
```

```
class Window:
  train_start: int
  train_end: int
  test_start: int
  test_end: int

def rolling_windows(start_ts, end_ts, train_days=180, test_days=60,
step_days=60) -> Iterable[Window]:
  # yield windows without leakage
  ...

def run_walk_forward(strategy_factory, dataset, param_grid):
  results = []
  for w in rolling_windows(dataset.start, dataset.end):
    train = dataset.slice(w.train_start, w.train_end)
    test  = dataset.slice(w.test_start, w.test_end)

    best_params = select_params_by_train_metric(train, strategy_factory,
param_grid)
    oos_metrics = backtest(test, strategy_factory(best_params))

    results.append({"window": w, "best_params": best_params, "oos":
oos_metrics})
  return results
```

Multiple-testing defense outline

If Lane A is pursued (large rule libraries), the pasted memo's requirement for multiple-testing control can be grounded with White's Reality Check or PBO: - White (2000) motivates testing the best model after specification search against a benchmark to avoid data snooping errors. [42]
- Bailey et al. propose PBO/CSCV explicitly for investment backtests. [43]

A pragmatic approach is: implement a "library run" where you evaluate rule families, then apply either Reality Check–style bootstrap (if you can implement it correctly) or PBO/CSCV to estimate how likely you are selecting an overfit rule.

Testing plan and CI/CD changes

Testing should expand in three layers:

- Unit tests:
- config-driven parameter propagation (risk fraction, slippage bps)
- candle downloader pagination correctness (limit caps) [31]
- cost model invariants (fees/slippage never negative, etc.)

- Integration tests:

- backtest engine replays a tiny synthetic bar set and yields deterministic metrics
- walk-forward windows don't overlap incorrectly and don't leak future data
- End-to-end:
- "paper run" can be started and stopped; run manifest and event counts are consistent with DB; safety guards persist across restart (already covered well). [38]

CI/CD (minimum): - Add a GitHub [45] Actions workflow that installs deps (including pytest), runs `pytest`, and optionally runs ruff/mypy. - Separate runtime deps from dev deps (e.g., `requirements.txt` vs `requirements-dev.txt`).

Legal, compliance, and security considerations

API key handling and credential security

The repo uses environment variables (`COINBASE_API_KEY`, `COINBASE_API_SECRET`) and does not embed secrets in code, which matches best practices. [27] [46]

However, moving toward any live trading increases the need to adopt the operator-grade controls recommended in Coinbase's security guidance: - Do not store keys in source control; use env vars or a secret manager. [46]
- Use IP allowlists and least-privilege permissions (especially avoid enabling transfer-like permissions unless essential). [47]
- JWTs expire quickly (2 minutes) and must be rotated; your WebSocket auth logic should be designed around that TTL. [48]

Rate limits and operational misuse risk

The Advanced Trade WebSocket has explicit rate limits (8 per second per IP) for connections/messages. [30] Expanding toward Lane C (order book data, more channels, more symbols) increases the risk of rate-limit bans or dropped connections unless you implement subscription consolidation, backpressure, and robust reconnect logic.

Financial and regulatory context

This repository currently fails fast on "live mode" and focuses on paper mode, which reduces immediate regulatory exposure but does not eliminate it if users repurpose it. [44]

If Lane B (derivatives carry/funding) becomes the target, the compliance and operational surface becomes materially larger: derivatives access may be jurisdiction-restricted and introduces liquidation/margin risk. The academic and policy literature treats "carry/basis" as a measurable phenomenon in crypto (e.g., BIS "Crypto carry"), but implementability depends on exchange/instrument access and risk controls. [40]

Strategy research integrity and “false discovery” risk

The pasted requirements strongly emphasize avoiding curve fitting. There is a real research risk that “trying many rules until something works” yields spurious edge—White’s Reality Check is explicitly motivated by this data snooping problem. [42] PBO/CSCV is similarly motivated by the observation that standard hold-out approaches can be misleading for backtests. [43]

If you pursue Lane A (rule libraries), you should treat multiple-testing control as a **first-order compliance requirement** for your own process, not an optional enhancement, because papers that claim profitability in large-rule settings explicitly implement such controls. [8]

---

[27] coinbase_adapter.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/coinbase_adapter.py

[2] main.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/main.py

[3] architecture.md
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/docs/system/architecture.md

[4] state_machine.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/state_machine.py

[5] roadmap_and_metrics.md
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/docs/project/roadmap_and_metrics.md

[6] [8] Technical trading and cryptocurrencies | Annals of Operations Research | Springer Nature Link
https://link.springer.com/article/10.1007/s10479-019-03357-1?utm_source=chatgpt.com

[7] Are simple technical trading rules profitable in bitcoin markets? - ScienceDirect
https://www.sciencedirect.com/science/article/pii/S1059056024003010?utm_source=chatgpt.com

[9] README.md
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/docs/README.md

[10] config.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/src/core/config.py

[11] config.yaml
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/config.yaml

[12] risk.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/risk.py

[13] requirements.txt
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/requirements.txt

[14] pyproject.toml
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/pyproject.toml

[15] paper_mode_implementation_spec.md
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/docs/paper_mode_implementation_spec.md

16 paper_mode_build_report.md
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/docs/paper_mode_build_report.md

17 market_data.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/market_data.py

18 bars.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/bars.py

19 aggregator.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/aggregator.py

20 strategy.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/strategy.py

21 db.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/db.py

22 journal.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/journal.py

23 execution.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/execution.py

24 adapters.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/adapters.py

25 safeguards.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/safeguards.py

26 events.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/events.py

28 Advanced Trade WebSocket Channels - Coinbase Developer Documentation
https://docs.cdp.coinbase.com/coinbase-app/advanced-trade-apis/websocket/websocket-channels

29 48 Advanced Trade WebSocket Authentication - Coinbase Developer Documentation
https://docs.cdp.coinbase.com/coinbase-app/advanced-trade-apis/websocket/websocket-authentication

30 Advanced Trade WebSocket Rate Limits - Coinbase Developer Documentation
https://docs.cdp.coinbase.com/coinbase-app/advanced-trade-apis/websocket/websocket-rate-limits

31 Get Product Candles - Coinbase Developer Documentation
https://docs.cdp.coinbase.com/api-reference/advanced-trade-api/rest-api/products/get-product-candles?utm_source=chatgpt.com

32 test_db.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_db.py

33 test_config.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_config.py

34 test_state_machine.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_state_machine.py

35 test_execution.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_execution.py

36 test_reconcile.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_reconcile.py

37 test_paper_mode.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_paper_mode.py

38 45 test_safeguards.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_safeguards.py

39 test_main_events.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/tests/test_main_events.py

40 Crypto carry
https://www.bis.org/publ/work1087.htm?utm_source=chatgpt.com

41 Price Discovery in Bitcoin: The Role of Limit Orders by Carol Alexander, Daniel F. Heck, Andreas Kaeck :: SSRN
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4150979&utm_source=chatgpt.com

42 A Reality Check for Data Snooping | The Econometric Society
https://www.econometricsociety.org/publications/econometrica/2000/09/01/reality-check-data-snooping?utm_source=chatgpt.com

43 The Probability of Backtest Overfitting by David H. Bailey, Jonathan Borwein, Marcos Lopez de Prado, Qiji Jim Zhu :: SSRN
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2326253&utm_source=chatgpt.com

44 config.py
https://github.com/Izayauh/coinbase-rtb-bot/blob/master/bot/config.py

46 47 Best Practices: API Security - Coinbase Developer Documentation
https://docs.cdp.coinbase.com/get-started/authentication/security-best-practices?utm_source=chatgpt.com